SOMAIYA
VIDYAVIHAR
K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

Name : Varad Eklare. **Roll no.: A-34 Batch: AS2**

Class : Second Year (SY) 2026-27 **Department : Information Technology**

Course code: ITXS37 **Subject: DSA**

Experiment No:3

Aim: Experiment No. 03 infix to postfix

Program

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>



#define MAX 100



char stack[MAX];

int top = -1;

void push(char c)

{

    if (top == MAX - 1) {
```

```
        printf("Stack Overflow\n");

        exit(1);

    }

    stack[++top] = c;

}



char pop(void)

{

    if (top == -1) {

        printf("Stack Underflow (invalid expression)\n");

        exit(1);

    }

    return stack[top--];
```

```
}

char peek(void)

{

    return (top == -1) ? '\0' : stack[top];

}

int isEmpty(void)

{

    return top == -1;

}
```

```
int precedence(char op)

{

   switch (op) {

      case '^': return 3;

      case '*':

      case '/':

      case '%': return 2;

      case '+':

      case '-': return 1;

      default : return 0;

   }

}
```

```
int isOperator(char c)

{

    return (c == '+' || c == '-' || c == '*' ||

            c == '/' || c == '%' || c == '^');

}









int isRightAssociative(char c)

{

    return c == '^';

}
```

```
void infixToPostfix(const char infix[], char postfix[])

{

    int i, k = 0;

    char c;

    for (i = 0; infix[i] != '\0'; i++) {

        c = infix[i];

        if (isspace((unsigned char)c))

            continue;

        if (isalnum((unsigned char)c)) {
```

```
    postfix[k++] = c;

}

else if (c == '(') {

    push(c);

}

else if (c == ')') {

    while (!isEmpty() && peek() != '('

        postfix[k++] = pop();

    if (isEmpty()) {

        printf("Mismatched parentheses\n");

        exit(1);

    }
```

```
    pop();

}

else if (isOperator(c)) {

    while (!isEmpty() && peek() != '(' &&

           (precedence(peek()) > precedence(c) ||

           (precedence(peek()) == precedence(c) &&

            !isRightAssociative(c)))) {

        postfix[k++] = pop();

    }

    push(c);

}

else {

    printf("Invalid character '%c' in expression\n", c);
```

```
        exit(1);

    }

}

while (!isEmpty()) {

    c = pop();

    if (c == '(') {

        printf("Mismatched parentheses\n");

        exit(1);

    }

    postfix[k++] = c;

}
```

```
    postfix[k] = '\0';

}

int main(void)

{

    char infix[MAX], postfix[MAX];

    printf("Enter a valid infix expression: ");

    fgets(infix, sizeof(infix), stdin);

    infix[strcspn(infix, "\n")] = '\0';

    infixToPostfix(infix, postfix);
```

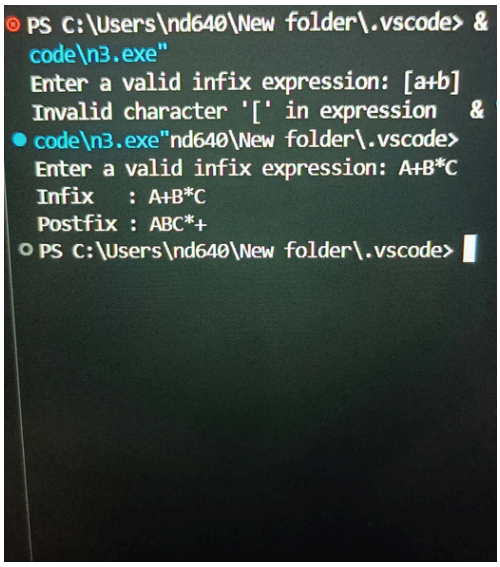
```
    printf("Infix   : %s\n", infix);

    printf("Postfix : %s\n", postfix);

    return 0;

}
```

Outcome:


PS C:\Users\nd640\New folder\.vscode> &
code\n3.exe"
Enter a valid infix expression: [a+b]
Invalid character '[' in expression &
code\n3.exe"nd640\New folder\.vscode>
Enter a valid infix expression: A+B*C
Infix : A+B*C
Postfix : ABC*+
PS C:\Users\nd640\New folder\.vscode>

Conclusion: SUCESSFULLY EXCUTED USING STRING